

COURSE INFORMATION

Course Code & Title	Academic Year / Term	Effective Start Date
IS115 Algorithms & Programming	AY2025-26 January Term	25/08/2025

Offering Unit / Department / School	Course Career
SCIS	Undergraduate

Course Units	Grading Basis
CU:	GRD - Graded

1

Course Description

A&P can be viewed as a first course in algorithms. Students will be trained to compute the time complexity of algorithms and compare algorithms using their Big-O time complexity. Besides coming up with the pseudocode of an algorithm to solve a given problem, students are expected to implement the pseudocode in Python for the project assignments. Common data structures such as stacks, queues, trees, graphs, heaps and hash maps will be covered. After students have gained a strong foundation in complexity and data structures, students will be introduced to heuristic approaches – specifically greedy and local search algorithms – that can be used to tackle computationally intractable problems.

Standard Learning Outcomes (for SOA only)

Course Learning Outcomes

Students will be trained in the following competencies:

1. Understanding how an algorithm solves a problem
 2. Explaining basic algorithms such as searching and sorting and applying them to solve problems
 3. Applying iteration or recursion to design an algorithm
 4. Analysing and comparing the complexities of different algorithms
 5. Improving the complexity of algorithms by designing more efficient solutions
 6. Demonstrating the ability to use fundamental data structures to solve problems
 7. Implementing an algorithm in a programming language such as Python
 8. Knowing the limits of computations, including concepts such as P, NP, NP-Complete, and NP-Hard
 9. Contrast optimal and approximation solutions
 10. Use heuristic strategies such as greedy algorithms to solve an intractable problem
1. Discover the science of computing
 2. Model problems and learn practical problem-solving techniques to tackle complex computational problems.
 3. Apply problem-solving techniques to develop more elegant and efficient programs
 4. Learn to write programs to represent and manipulate with complex data objects
 5. Understand the challenge of scale, not only in dealing with large data sets, but also in appreciating the nature of computing and computability
 6. Derive the Big-O time complexity of an algorithm, and compare two algorithms in terms of time complexity.

SMU Graduate Learning Outcomes

Disciplinary Knowledge, Critical thinking & problem solving, Collaboration and leadership, Communication, Self-directed learning, Resilience

Discipline Specific Competencies

Applications Development, Formal Proof Construction, Algorithm Analysis, Combinatorial Decision-making, Computational Modelling

Course Area(s)

AREA - Accounting Data and Analytics Electives, AREA - Accounting Electives, AREA - Accounting Options, AREA - Business Options, AREA - Computing & Law Core (Intake 2024 onwards), AREA - Financial Forensics Electives, AREA - Grad Req - Dig Tech/Data Ana (Intake 2024 onwards), AREA - Information Systems Core (Intake 2024 onwards), AREA - Software Engineering Core (Intake 2024 onwards), AREA - Technology Studies Cluster, RQCP - IS111/CS101/COR-IS1704, RQCX - COR-IS1702, RQCX - CS201, ENRL - Open to Undergraduates only

Simple Requisites

IS111/CS101/COR-IS1704 - Pre-req

Type

Prerequisite

-

Complete ANY of the following Courses:

- IS111 - Introduction to Programming
- CS101 - Programming Fundamentals I
- COR-IS1704 - Computational Thinking and Programming

-

IS115/ COR-IS1702/CS201 - Mutually Exclusive

Type

Antirequisite

-

Enroll in the following Courses:

- COR-IS1702 - Computational Thinking
- CS201 - Data Structures and Algorithms

-

CLASS SECTION INFORMATION

Instructor(s)

MAI ANH TIEN -

Course Instructor(s)

Faculty: MAI Anh Tien (course coordinator), MOK Heng Ngee, ZHANG Zhiyuan

Instructor:

Instruction Mode

In Person

Consultations Assistants

-

and Teaching

Office Location

-

Assessment Components (New)

Category Class Participation and Activities	Description Class Participation	Weightage 20
Assessment Period (for Proctored Assessments) -		

Category Class Participation and Activities	Description Hackathon	Weightage 20
Assessment Period (for Proctored Assessments) -		

Category Proctored Assessments	Description Mid-term Exam	Weightage 20
Assessment Period (for Proctored Assessments) -		

Category Final Exam	Description -	Weightage 40
Assessment Period (for Proctored Assessments) -		

Instructional Methods and Expectations

Upon completion of the course, students will be able to:

1. Discover the science of computing (or How to think like a Computer Scientist)
2. Model problems and learn practical problem-solving techniques to tackle complex computational problems (beyond what a spreadsheet is capable of solving)
3. Apply problem-solving techniques to develop more elegant and efficient programs
4. Learn to write programs to represent and manipulate with complex data objects
5. Understand the challenge of scale, not only in dealing with large data sets, but also in appreciating the nature of computing and computability
6. Derive the Big-O time complexity of an algorithm, and compare two algorithms in terms of time complexity.

Competencies

Students will be trained in the following competencies:

1. Understanding how an algorithm solves a problem
2. Explaining basic algorithms such as searching and sorting and applying them to solve problems
3. Applying iteration or recursion to design an algorithm
4. Analysing and comparing the complexities of different algorithms
5. Improving the complexity of algorithms by designing more efficient solutions
6. Demonstrating the ability to use fundamental data structures to solve problems
7. Implementing an algorithm in a programming language such as Python
8. Knowing the limits of computations, including concepts such as P, NP, NP-Complete, and NP-Hard
9. Contrast optimal and approximation solutions
10. Use heuristic strategies such as greedy algorithms to solve an intractable problem

Weekly Lesson Plan

Week	Topic	Remarks
1	Introduction to Algorithms, Counting	
2	Complexity	
3	Iteration & Decomposition (search, sort)	
4	Recursion	
5	Data Structures 1: Stacks & Queues	
6	Data Structures 2: Trees	Mid-term Exam
7	Data Structures 3: Graphs	
8	Recess	
9	Graph Algorithms	
10	Data Structures 4: Heaps & Hashmaps More algo exercises	
11	Hackathon (no lesson)	Hackathon
12	Heuristics (greedy algorithms, local search)	
13	Limits of Computing, Course Review	
14	Study week (no lesson)	
15/16	Exam Period	Final Exam

Recommended Textbooks and Readings

Main Readings:

- [JSC] Conery, John S., Explorations in Computing: An Introduction to Computer Science and Python Programming, CRC Press, 2015. Chapters 1-6, 12. Online ebook is available at the SMU library: https://search.library.smu.edu.sg/permalink/65SMU_INST/1ba19kd/cdi_askewsholts_vlebooks_9781466572454
- [KHR] Rosen, Kenneth H., Discrete Mathematics and Its Applications, 7th Edition, McGraw-Hill, 2012. Chapter 6 Counting.
- [Handout] Supplementary Handout on Fundamental Data Structures provided by instructors, 2012. Released through eLearn.

Optional Reading:

- [PC] Prichard and Carrano, Data Abstraction and Problem Solving with Java: Walls and Mirrors, 3rd Edition, 2011

Tools:

- Any Python 3 IDE, or Google Colab (online, via browser)

Other Information

-

UNIVERSITY POLICIES

Academic Integrity

All acts of academic dishonesty (including, but not limited to, plagiarism, cheating, fabrication, facilitation of acts of academic dishonesty by others, unauthorized possession of exam questions, or tampering with the academic work of other students) are serious offences.

All work (whether oral or written) submitted for purposes of assessment must be the student's own work. Penalties for violation of the policy range from zero marks for the component assessment to expulsion, depending on the nature of the offence.

When in doubt, students should consult the instructors of the course. Details on the SMU Code of Academic Integrity may be accessed at <https://smu.sharepoint.com/sites/oasis/SitePages/DOS-WKLSWC/UCSC.aspx>.

Responsible Use of AI

The University recognises the importance of equipping students with the skills to navigate an AI-driven world. While AI tools may support learning, they must be used judiciously and responsibly. Students must not present AI-generated output as their own work.

Each course may set specific guidelines on the use of AI in assessments. Students are required to follow these guidelines carefully and should seek clarification from the instructors if uncertain.

Misuse of AI constitutes a breach of academic integrity and will be subject to the same penalties as other acts of academic dishonesty.

Copyright Notice

Please note that all course materials are meant for personal use only, namely for the purposes of teaching, studying and research. You are strictly not permitted to make copies of or print additional copies or distribute such copies of the course materials or any parts thereof, for commercial gain or exchange.

For the full copyright notice, please visit <https://researchguides.smu.edu.sg/copyright>.

Accessibility

SMU strives to make learning experiences accessible for all. Students who anticipate or experience physical or academic barriers due to disability are encouraged to inform the course instructor as soon as possible. Students may also contact the university's disability services team if they have questions or concerns about academic provisions: accessibility@smu.edu.sg. Please be aware that the accessible tables in our seminar room should remain available for students who require them.

Digital Readiness for Teaching and Learning (DRTL)

As part of emergency preparedness, instructors may conduct lessons online via the Zoom platform during the term, to prepare students for online learning. During an actual emergency, students will be notified to access the Zoom platform for their online lessons. The class schedule will mirror the current face-to-face class timetable unless otherwise stated.